Authentication, NSS and PAM

Computing Systems Security · MCS and MEI · University of Minho · 2026-27

Vítor Francisco Fonte <vff@di.uminho.pt> · 2026-08-19

NSS and PAM: Lecture Overview

Process Credentials: the identity every access decision is made against, and how a process acquires it

Name Resolution: which sources define an account, and what changes when the answer comes over a network

PAM: module types, control flags, and one authentication traced through the stack Ubuntu ships

Hardening the Stack: lockout and password quality, and the new problem each of them introduces

Into the Lab: where PAM ends, and the two answers Weeks 4 and 5 give instead

The tab on the right edge of each slide shows the tier of the slide.

Lecture Core: covered in session. Lab Core: connects to laboratory work. Study Support: post-session review. Reference: lookup material.

Identity Comes First

- Every access decision on this machine is the same question: **who is asking?**
- The asker is never a person. It is a **process**, carrying an identity the kernel assigned it
- Authentication is how that identity gets attached. Authorisation is what is then done with it
- Get identity wrong and every permission, capability and policy later in the course is decided for the wrong subject

Authentication answers who. Authorisation answers what they may do. Conflating them is the commonest confusion in this field.

Real, Effective, Saved

```
$ id -u ; id -un
1000
student

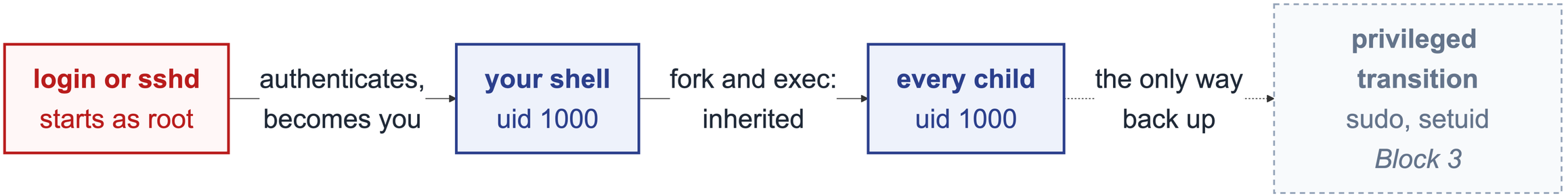
$ grep ^Uid: /proc/self/status
Uid:   1000   1000   1000   1000
       real   eff    saved  fs
```

- **Real** is who you are, the identity the session was established with, and what the system bills
- **Effective** is what you are right now: every permission check reads this one
- **Saved** is what you may return to, which is how a program drops privilege and takes it back
- The fourth is the filesystem UID, a Linux detail that follows the effective UID in practice
- All four read 1000 here because nothing has transitioned. Week 6 is where they start to differ

One process, several identities at once. The check that matters uses the effective one.

How a Process Gets Its Identity

- A privileged process authenticates you and then **becomes** you: that is what login does
- After that, identity is **inherited**. Every `fork` and every `exec` carries it forward unchanged
- Going back up requires a deliberate, privileged transition, and that is the only route



Identity is set once and inherited thereafter. Everything interesting happens at the two ends of this chain.

The Account List and the Hashes

```
$ stat -c '%A %U %G %n' /etc/passwd /etc/shadow
-rw-r--r-- root root /etc/passwd
-rw-r----- root shadow /etc/shadow

$ grep ^student: /etc/passwd
student:x:1000:1001:...:/bin/bash
        ^ the hash used to live here

$ sudo grep ^student: /etc/shadow
student:$y$j9T$pYFNPRO...:20318:0:99999:
        ^ id, salt, hash
```

- `/etc/passwd` is world-readable: every `ls -l` has to turn a number into a name
- The `x` is a historical scar: the hash was once in that field, readable by anyone on the machine
- `/etc/shadow` is not world-readable, so an unprivileged foothold cannot carry the hashes away
- The split is a trust boundary, and the boundary is enforced by a file mode

Compare Lecture 2's read bit. The same mechanism, used deliberately this time.

Hashes at Rest

- **Never plaintext**, so that a stolen file is not a stolen password
- **Salted**, so equal passwords get different hashes, and each account has to be cracked separately
- **Slow** on purpose, so that guessing costs the attacker time the login path can still afford
- This image uses **yescrypt**, which is why the field begins y. The mathematics belongs to Data Security

Hashing does not protect a weak password. It buys time, and only against somebody who already has the file.

Study Support

Name Resolution

Before anything checks your password, something has to decide that you exist. Who decides?

Which Sources Define an Account

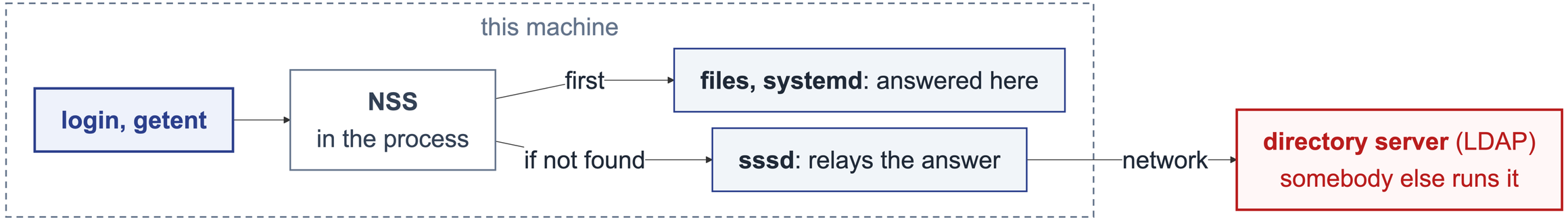
```
$ grep -E '^(passwd|shadow):' /etc/nsswitch.conf
passwd:   files systemd
shadow:   files systemd
```

- The **Name Service Switch** (NSS) answers first: does this account exist, and who says so
- `/etc/nsswitch.conf` is the policy: an ordered list of sources, one list per database
- On the course image the account databases read `files systemd`, so every answer is local
- One line of configuration decides whether your machine's account list is actually yours

Authentication is the second question. The first is resolution, and it is answered by whoever this file names.

Files, or Somewhere Else

- `files` and `systemd` answer from this machine. `sss` asks `sssd`, which answers from cache or asks a directory
- Organisations centralise for good reasons: one account, one password change, one revocation
- The cost is that identity data now crosses a network, onto the wire and onto the directory's operator
- Neither is safer by default. Local scatters the risk, central concentrates it, and both are choices



The moment identity is centralised, the directory joins the trust base of every machine that consults it.

getent Is the Lens

```
$ getent passwd student
student:x:1000:1001:...:/bin/bash

$ getent passwd 0
root:x:0:0:root:/root:/bin/bash
```

- `getent` asks NSS the same question the system asks, through the same sources, in the same order
- The difference between reading `/etc/passwd` and knowing what the machine will answer
- If `getent` returns an account that `grep` on the file does not, the answer came from somewhere else
- Use it whenever you need what is true rather than what a file appears to say

Lab Core

Ask the resolver, not the file. On a machine with a directory source they are different questions.

When the Directory Is the Target

- Compromise the **directory**, the server with every account, and you compromise **every** machine that trusts it
- The same holds for availability: if the directory cannot be reached, its accounts log in nowhere
- This is the block's first real trade-off. Local means many places to manage, central means one place to lose
- Weeks 4 and 5 give two different answers: delegate the decision, or verify a statement signed in advance

Centralised identity concentrates the convenience and the blast radius in the same place. Neither is free.

PAM

Every service on this machine needs to check a password. Why does none of them contain the check?

The Problem PAM Solves

- Without it, every service that authenticates would carry its own copy of the password check
- Changing policy would then mean changing, testing and shipping every one of them, and missing one
- PAM, **Pluggable Authentication Modules**, inverts it: the service asks a library, the administrator composes modules
- One consequence worth keeping: the service no longer knows **how** you were authenticated, only **whether** you were

A policy seam, wired into a real system. This is Lecture 1's policy-and-mechanism split, in configuration form.

Four Module Types

- **auth**: prove identity. A password, a key, a token, a fingerprint
- **account**: may this account be used right now. Expiry, time windows, membership
- **password**: govern credential **changes**
- **session**: set up and tear down what a session needs. Limits, environment, mounts, logging
- The lines of one type form a stack, and each stack runs at its own moment

Four types, four different questions. Putting a rule in the wrong stack is a real and common bug.

How a Stack Composes

- **required**: must succeed, and the rest of the stack still runs. The failure is reported at the end
- **requisite**: must succeed, and failure returns immediately
- **sufficient**: success ends the stack successfully, unless something required has already failed
- **optional**: the result is ignored unless this is the only module in the stack
- `[success=1 default=ignore]` is the general form: on success skip one line, otherwise ignore this result

Order is policy. The same four modules in a different order are a different security decision.

Inside /etc/pam.d

```
$ cat /etc/pam.d/sudo
session  required  pam_limits.so
session  required  pam_env.so readenv=1 user_readenv=0
session  required  pam_env.so readenv=1 envfile=...
@include common-auth
@include common-account
@include common-session-noninteractive
```

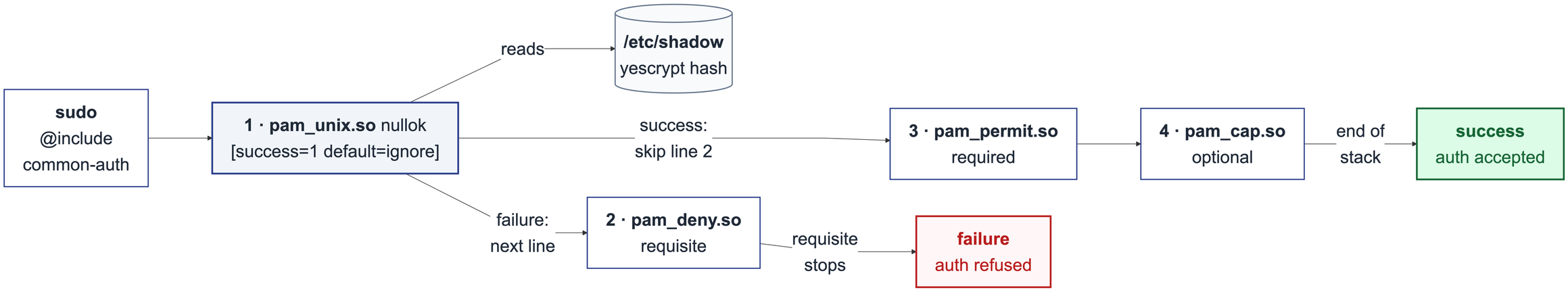
- One file per service, in `/etc/pam.d/`
- The `common-*` files are the shared stacks that most services pull in with `@include`
- So `sudo` holds almost no authentication policy of its own. It inherits the machine's
- Change `common-auth` and you have changed `sudo`, `login`, `sshd` and everything else at once

That include is why a one-line edit can lock every service on the machine out at the same moment.

Lab Core

One Authentication, Traced

- `sudo` includes `common-auth`, and that is where the decision is actually made
- The `pam_unix.so` line ends in `nullok`, a permissive default that arrived with the profile
- It permits an **empty** password field, which in `/etc/shadow` is not the same as * or !
- On this image: * on eighteen accounts, ! on ten, both on five, one real hash, and none empty



This is the whole of local password authentication: a handful of lines, every one of them replaceable.

Profiles, Not Hand Edits

- On Ubuntu the `common-*` files are **generated**, from profiles in `/usr/share/pam-configs/`
- A module that comes with a profile is enabled in `pam-auth-update`, not by editing the generated file
- A module added inside the generated block stops profile updates until the local changes are overridden
- In the laboratory you will hand-edit `/etc/pam.d/sudo`, which is a service file and not a generated one

Know which files are generated. On this machine common-auth is, and sudo is not.

Study Support

When the Stack Breaks

- Misname a module, or remove one, and the stack cannot be loaded
- PAM then **fails closed**: the authentication is refused rather than allowed
- That is the only defensible default. An authentication stack that failed open would be an open door with a log entry
- The cost is real and is yours to carry: a mistake in a shared stack locks everyone out, including you

Open a root shell before you touch an auth file. That shell is the difference between a mistake and a reinstall.

Hardening the Stack

Two modules, two risks reduced, and two new problems created. Which trade would you accept?

Slowing Down the Guesser

- `pam_faillock` counts failed authentications and refuses the account past a threshold
- It **brackets** the password check: once **before** `pam_unix.so`, and twice **after** it
- Before, it refuses an account already locked; after, a failure adds one and a success clears the count
- A final `pam_deny.so` fails the rest closed. You write this stack in the laboratory

Three failures and the account is refused. Before you choose the threshold, decide what that costs, and to whom.

Quality, in the Right Stack

- This image ships `password requisite pam_pwquality.so retry=3`, in the **password** stack
- Why the **password** stack and not **auth**? The laboratory asks you, so hold the question
- No thresholds are set, so the built-in floor applies: **length 8**, no class rules, dictionary on
- It can enforce length, classes and dictionary checks, all of which measure a password's **shape**
- It cannot know the password is already in a breach corpus, or reused on another site. Shape is not secrecy
- The laboratory has you add class rules to watch one fire. That is a demonstration, not advice

Quality rules raise the floor. They say nothing about whether this particular password is already known to somebody.

Where PAM Ends

- PAM authenticates a **local** session, against credentials this machine can reach
- With a password, the machine that asks is also the machine that receives it, even when a directory checks it
- Week 4 delegates the decision to an identity provider: the service that asks gets a token, not the password
- Week 5 verifies a statement signed in advance, so nobody has to be asked anything at the moment of login

The same question every week: where does the trust live? With PAM, wherever its modules look, and a password typed here is received here.

Today in the Laboratory

Lab 3 breaks and repairs sudo's authentication. Snapshot first, and keep a root shell open.

- Read your own credentials and this machine's stack: `id`, `/proc/self/status`, `nsswitch.conf`, `/etc/pam.d/`
- Break `sudo`'s auth deliberately, watch it fail closed, and repair it from the root terminal
- Write a lockout stack and watch the counter climb, then add a quality rule and see a weak password refused
- Only **P1-P2 and your reflection** go into **lab report 2**; the exercises are practice for Lab 5

Break sudo, never login. Choosing the victim carefully is the whole safety argument of this session.

Lab Core

Three Answers to One Question

- **This week.** The password reaches the machine, which checks it locally or against a directory
- **Week 4.** An identity provider decides and the machine trusts a token. Scales, and concentrates risk at the provider
- **Week 5.** An authority signs in advance and the machine verifies. Nobody is asked at login time
- One question, three answers, and each of them fails differently. That comparison is the block's deliverable

Nobody removes trust. The engineering choice is where to place it, and what happens on the day it fails.

Reading and Reference, All Optional

- **The Linux-PAM System Administrators' Guide**. The authoritative description of module types, control flags and stack semantics. https://github.com/linux-pam/linux-pam/tree/master/doc
- **pam.conf(5), pam_faillock(8), pam_pwquality(8), nsswitch.conf(5)**. The manual pages behind every line quoted in this lecture. Read them on the machine.
- **Stallings and Brown, Computer Security: Principles and Practice, 4th ed., 2018**. Chapter 3, user authentication, for the password and hashing background.
- **W. Du, Computer Security: A Hands-on Approach, 3rd ed., 2022**. The set-UID and authentication chapters, closest to this unit's laboratory style.
- **Ubuntu, pam-auth-update(8)**. How the common-* stacks are generated, and what happens to a hand edit.

Reference

Key Takeaways

- Every access decision starts from a process identity, and that identity has three parts rather than one
- NSS decides who answers whether an account exists, and the answer can come from off this machine
- PAM is composed policy: types, order and control flags, and the order is itself a security decision
- It fails closed, which is why a mistake in an auth file is an outage and not a breach
- Hardening is a trade: a control that blocks one attack can open another, and the laboratory shows you which

Ask of any authentication system: who is asked, what is compared, and what happens when no answer can be obtained.